

A COMPLETE THEORETICAL & PRACTICAL
GUIDE

Domain-Driven Design with .NET

Clean Architecture · OOP Principles · Design Patterns ·
CQRS

Full Working Code Walkthrough (ASP.NET Core + EF
Core + MediatR)

Strategic &
Tactical DDD

Clean
Architecture
Layers

Design
Patterns
Catalog

End-to-
End
Project

Table of Contents

Chapter 1 — Introduction to Domain-Driven Design

Chapter 2 — Strategic Design (Ubiquitous Language, Bounded Contexts, Context Mapping, Subdomains)

Chapter 3 — Tactical Design Building Blocks (Entities, Value Objects, Aggregates, Domain Events, Domain Services, Repositories, Factories, Specifications)

Chapter 4 — OOP Principles Inside DDD (Encapsulation, Abstraction, Inheritance, Polymorphism)

Chapter 5 — Clean Architecture (Layers, the Dependency Rule, why DDD + Clean Architecture fit together)

Chapter 6 — Design Patterns Used Across the Architecture (Repository, Unit of Work, Factory, Specification, Strategy, Mediator/CQRS, Decorator, Observer, Builder, Adapter)

Chapter 7 — CQRS and MediatR in Practice

Chapter 8 — Full Practical Project: Online Store Order Management

8.1 Solution & Folder Structure

8.2 Domain Layer (Entities, Value Objects, Aggregate Root, Events, Specifications)

8.3 Application Layer (CQRS Commands/Queries, Handlers, Validation, DTOs)

8.4 Infrastructure Layer (EF Core, Repositories, Unit of Work, DI)

8.5 Presentation Layer (API Controllers, Middleware, Program.cs)

Chapter 9 — Unit Testing the Domain Layer

Chapter 10 — When (Not) to Use DDD

Chapter 11 — Common Pitfalls & Anti-Patterns

Chapter 12 — Summary & Further Reading

Chapter 1 — Introduction to Domain-Driven Design

1.1 What Is Domain-Driven Design?

Domain-Driven Design (DDD) is an approach to software development introduced by Eric Evans in his 2003 book *"Domain-Driven Design: Tackling Complexity in the Heart of Software."* It is **not a framework, not a technology, and not a specific architecture** — it is a **set of principles and patterns** that help you build software whose structure and language directly reflect a deep, evolving understanding of the business (the "domain") it serves.

The core idea is simple to state but hard to master: **the complexity of most software systems lives in the business rules, not in the technology.** Databases, web frameworks, and cloud infrastructure are solved problems. What is hard is correctly modeling concepts like "an order can only be shipped once payment is confirmed" or "a loan cannot be approved if the customer's debt-to-income ratio exceeds 40%." DDD gives you a disciplined way to capture that logic in code so it stays correct as the system grows.

Core Definition

DDD is the practice of building a software model that mirrors a real business domain, using a language shared by developers and domain experts (the **Ubiquitous Language**), and protecting business rules inside well-defined objects instead of scattering them across services, controllers, and database scripts.

1.2 The Problem DDD Solves

Most systems do not fail because of bad technology choices. They fail because business logic becomes tangled, duplicated, and impossible to change safely. This usually happens when:

- Business rules are spread across controllers, services, stored procedures, and UI validation — so nobody knows where "the truth" lives.
- Models are **anemic**: classes are just bags of public properties (`get`; `set`;) with no behavior, and all logic sits in "Manager" or "Service" classes that manipulate them from the outside.
- A single generic data model is reused for every part of the business, forcing unrelated concepts (e.g. "Customer" in Sales vs. "Customer" in Support) to share one bloated class.
- Developers and business stakeholders use different vocabularies, so requirements get lost or misinterpreted in translation.

DDD attacks all four problems directly: it pushes behavior *into* the objects that own the data (rich domain model), it splits large systems into smaller, independently-modeled areas (Bounded Contexts), and it forces a shared, precise vocabulary (Ubiquitous Language) between code and business.

1.3 DDD Is Split Into Two Halves

Strategic Design	Tactical Design
High-level. About how to divide a large system into meaningful, loosely-coupled parts (Bounded Contexts) and how those parts talk to each other. Answers "what are the pieces, and where are the boundaries?"	Low-level. About how to build the code inside one Bounded Context using building blocks such as Entities, Value Objects, Aggregates, Domain Events, Repositories and Domain Services. Answers "how do I write this class correctly?"

Chapter 2 covers Strategic Design; Chapter 3 covers Tactical Design in depth, with C# examples.

1.4 Where .NET and Clean Architecture Fit In

DDD tells you *how to think about and model the business*. It does not tell you how to organize projects, which framework to use, or how a web request reaches your domain code. That is the job of an **architecture**. **Clean Architecture** (a layered architecture popularized by Robert C. Martin, closely related to Hexagonal/Ports-and-Adapters and Onion Architecture) is the architecture most commonly paired with DDD in the .NET ecosystem because it enforces exactly the separation DDD needs: the domain model must never depend on infrastructure, databases, or frameworks. Chapter 5 explains this relationship in detail, and Chapter 8 builds a full solution combining both.

How to Read This Guide

Read Chapters 1–7 in order — each concept builds on the last. Chapter 8 is the practical anchor: every pattern discussed earlier is applied inside one real, coherent example (an Order Management module for an online store), with full C# code and inline comments explaining *why*, not just *what*.

Chapter 2 — Strategic Design

2.1 Ubiquitous Language

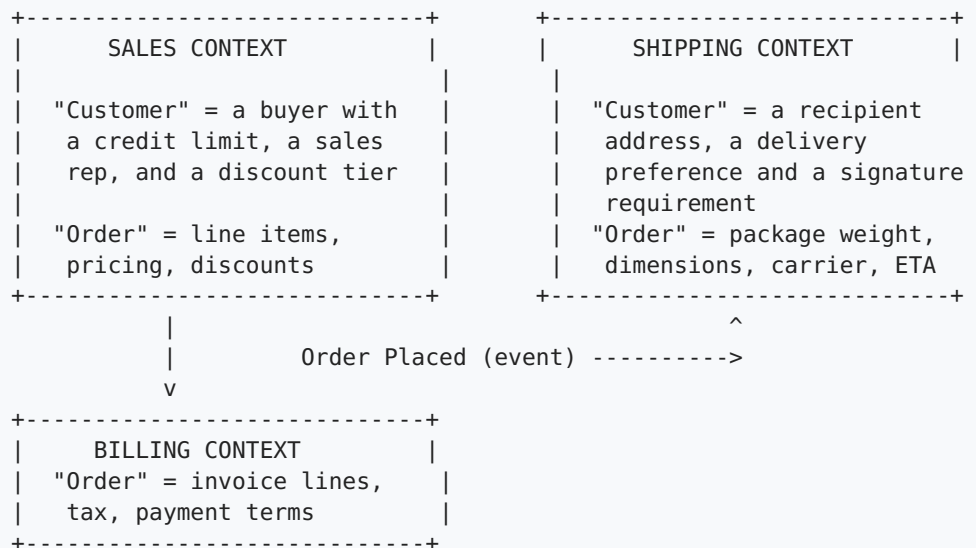
The **Ubiquitous Language** is a shared vocabulary built jointly by developers and domain experts (product owners, business analysts, actual users of the system). Every term in this language must have one precise meaning, and — critically — **that exact term must appear in the code**: in class names, method names, and even in conversation and documentation.

Example

If the business says "an order is *placed*, then *confirmed*, then *shipped*," the code should contain a class named `Order` with methods named `Place()`, `Confirm()`, `Ship()` — not generic names like `UpdateStatus(int status)`. The moment code and business language diverge, translation errors and bugs creep in.

2.2 Bounded Context

A **Bounded Context** is an explicit boundary (usually a module, a service, or even a separate deployable application) inside which a specific model and its Ubiquitous Language apply *consistently*. Outside that boundary, the same word can mean something completely different.



Trying to build one giant `Customer` or `Order` class that satisfies Sales, Shipping, and Billing simultaneously is a common and costly mistake — it produces a bloated class riddled with nullable, context-irrelevant fields and conditional logic. DDD instead encourages **one small, precise model per Bounded Context**, connected through integration events or well-defined APIs rather than a shared database table.

2.3 Subdomains: Core, Supporting, Generic

Before drawing Bounded Contexts, it helps to classify the business into subdomains by strategic importance:

Subdomain Type	Description	Investment Strategy
Core Domain	The part that gives the business its competitive advantage (e.g., a dynamic pricing engine for a logistics company).	Model it with full DDD rigor; write it yourself; invest your best developers here.
Supporting Subdomain	Necessary for the business but not a differentiator (e.g., a custom notification formatter).	Build it simply; DDD rigor optional.
Generic Subdomain	Solved problems common to every company (e.g., authentication, payment processing).	Buy or use an existing library/service — do not reinvent it.

2.4 Context Mapping

Once Bounded Contexts are identified, **Context Mapping** defines how they relate and integrate. Common patterns include:

- **Shared Kernel** — two contexts explicitly share a small piece of model/code (used sparingly — it creates coupling).
- **Customer–Supplier** — one context (supplier) provides data/services that the other (customer) depends on; the supplier's team should consider the customer's needs.
- **Conformist** — the downstream context simply accepts the upstream model as-is, with no influence over it (common when integrating a third-party API).
- **Anti-Corruption Layer (ACL)** — the downstream context builds a translation layer that converts the upstream model into its own clean model, preventing external concepts from leaking in. This is the pattern used most often in .NET when integrating legacy systems or third-party services.
- **Open Host Service / Published Language** — a context exposes a well-documented API and a standard data format (e.g., a versioned REST API and DTO contracts) for many consumers.

Practical takeaway for .NET solutions

In a typical .NET solution, each Bounded Context often becomes its own solution (or at least its own set of Domain/Application/Infrastructure projects), potentially its own microservice and database. Communication between contexts happens through integration events (e.g., via a message broker like RabbitMQ/Azure Service Bus) or synchronous APIs guarded by an Anti-Corruption Layer — never through direct shared database access.

Chapter 3 — Tactical Design Building Blocks

Tactical patterns are the concrete building blocks you use to implement a Bounded Context in code. This chapter explains each one theoretically, then shows a real C# implementation with comments. All examples use plain C# (no external framework needed for the domain layer itself — this is intentional, and explained in Chapter 5).

3.1 Entities

An **Entity** is an object defined not by its attributes, but by a **thread of continuity and identity**. Two entities with identical property values are still different objects if their identity (usually an **Id**) differs. A **Customer** named "John Smith" today is the same entity even after he changes his name, address, or email — because his **Id** never changes.

- Has a unique identifier (**Id**).
- Is mutable over its lifetime; equality is based on identity, not field values.
- Owns behavior and enforces its own invariants (business rules that must always be true).

```

// A generic base class shared by every Entity in the Domain layer.
// Centralizing identity + equality logic here avoids repeating it in every entity.
public abstract class Entity
{
    // The identity of the entity. 'protected set' means only this class
    // (and derived classes) can set it -- never from outside (encapsulation).
    public Guid Id { get; protected set; }

    protected Entity()
    {
        // New entities generate their own identity; the database
        // does not need to be the source of identity in DDD.
        Id = Guid.NewGuid();
    }

    protected Entity(Guid id)
    {
        Id = id;
    }

    // Entities are equal when their identities are equal --
    // NOT when their field values match (unlike Value Objects, see 3.2).
    public override bool Equals(object? obj)
    {
        if (obj is not Entity other) return false;
        if (ReferenceEquals(this, other)) return true;
        if (GetType() != other.GetType()) return false;
        return Id == other.Id;
    }

    public override int GetHashCode() => Id.GetHashCode();

    public static bool operator ==(Entity? a, Entity? b)
    {
        if (a is null && b is null) return true;
        if (a is null || b is null) return false;
        return a.Equals(b);
    }

    public static bool operator !=(Entity? a, Entity? b) => !(a == b);
}

```

3.2 Value Objects

A **Value Object** (VO) has **no identity**. It is defined entirely by the combination of its values, and it should be **immutable** — once created, it cannot change. Two Value Objects with the same values are considered completely interchangeable and equal. Examples: `Money`, `Address`, `Email`, `DateRange`.

Why Value Objects matter so much

Value Objects are where DDD eliminates an entire class of bugs. Instead of passing a raw `decimal` around for money (and risking mixing currencies, or accidentally adding a negative price), you wrap it in a `Money` type that *guarantees* its own correctness everywhere it appears. Validation happens exactly once — inside the constructor — instead of being repeated (or forgotten) in ten different places across the codebase.

```

// A Value Object representing money in a specific currency.
// 'record' gives us built-in value-based equality (matches VO semantics)
// and immutability (init-only properties) for free in modern C#.
public sealed record Money
{
    public decimal Amount { get; }
    public string Currency { get; }

    // The constructor is the ONLY place where a Money value can be created,
    // and it enforces the invariant: money can never be negative, and
    // currency must always be a valid 3-letter ISO code.
    public Money(decimal amount, string currency)
    {
        if (amount < 0)
            throw new ArgumentException("Money amount cannot be negative.", nameof(amount));

        if (string.IsNullOrEmpty(currency) || currency.Length != 3)
            throw new ArgumentException("Currency must be a 3-letter ISO code.", nameof(currency));

        Amount = amount;
        Currency = currency.ToUpperInvariant();
    }

    // Behavior lives on the Value Object itself -- this is what makes the
    // model "rich" instead of "anemic". Arithmetic also protects the
    // invariant that you cannot silently add two different currencies.
    public Money Add(Money other)
    {
        if (other.Currency != Currency)
            throw new InvalidOperationException(
                $"Cannot add {other.Currency} to {Currency}.");

        return new Money(Amount + other.Amount, Currency);
    }

    public Money Multiply(int factor) => new(Amount * factor, Currency);

    public static Money Zero(string currency) => new(0, currency);

    public override string ToString() => $"{Amount:0.00} {Currency}";

    // NOTE: because this is a 'record', C# auto-generates value-based
    // Equals()/GetHashCode() -- two Money instances with the same Amount
    // and Currency are automatically equal. That's exactly VO semantics.
}

```

3.3 Aggregates and Aggregate Roots

An **Aggregate** is a cluster of related Entities and Value Objects that must be treated as a single, consistent unit for the purpose of data changes. Every Aggregate has exactly one **Aggregate Root** — the only Entity inside the cluster that outside code is allowed to reference or hold a direct pointer to.

The Aggregate Rule

Outside code must **never** reach into an Aggregate and modify an internal Entity directly (e.g. `order.Items[0].Quantity = 5`). All changes must go through a method on the Aggregate Root (e.g. `order.ChangeItemQuantity(itemId, 5)`), so the root can validate the change against every business rule (invariant) that spans the whole cluster before allowing it.

Why does this matter? Consider an `Order` aggregate containing many `OrderLine` entities. A business rule might say "an order's total value cannot exceed the customer's credit limit." If external code could edit an `OrderLine` directly, that rule could never be reliably enforced — there is no single place responsible for checking it. By forcing all changes through the `Order` root, the rule lives in exactly one place and can never be bypassed.

- Aggregates are the **transaction boundary**: one aggregate = one unit of consistency, saved atomically in one database transaction.
- Keep aggregates **small**. A common mistake is designing one giant aggregate (e.g. `Customer` containing every order, every payment, every ticket ever created). Large aggregates cause performance problems and concurrency conflicts. Reference other aggregates by **Id only**, not by object reference.
- Only the Aggregate Root has a repository (see 3.6). Internal entities are never fetched or saved independently.

```

// =====
// AGGREGATE ROOT: Order
// Encapsulates OrderLine entities and enforces every invariant
// that spans the whole cluster.
// =====
public sealed class Order : Entity, IAggregateRoot
{
    private readonly List<OrderLine> _lines = new();

    // Expose a READ-ONLY view of internal entities.
    // External code can look, but can never Add/Remove/Modify directly --
    // that would bypass invariant checks. This is encapsulation in action.
    public IReadOnlyCollection<OrderLine> Lines => _lines.AsReadOnly();

    public Guid CustomerId { get; private set; }
    public OrderStatus Status { get; private set; }
    public Money Total { get; private set; }

    // Domain Events raised by this aggregate during this "session" --
    // collected here and dispatched AFTER a successful save (see 3.4).
    private readonly List<IDomainEvent> _domainEvents = new();
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();

    // Private constructor: forces creation through the static factory
    // method below, so an Order can never exist in an invalid state.
    private Order(Guid customerId, string currency)
    {
        Id = Guid.NewGuid();
        CustomerId = customerId;
        Status = OrderStatus.Draft;
        Total = Money.Zero(currency);
    }

    // FACTORY METHOD (see Chapter 6.3) -- the single, controlled entry
    // point for creating a valid new Order.
    public static Order Create(Guid customerId, string currency)
    {
        if (customerId == Guid.Empty)
            throw new ArgumentException("A customer id is required.", nameof(customerId));

        var order = new Order(customerId, currency);

        // Raise a domain event: other parts of the system (or other
        // Bounded Contexts) can react to "an order was created"
        // WITHOUT the Order aggregate knowing or caring who is listening.
        order._domainEvents.Add(new OrderCreatedEvent(order.Id, customerId));
        return order;
    }

    // Business behavior lives HERE, not in an external "OrderService".
    public void AddLine(Guid productId, string productName, Money unitPrice, int quantity)
    {
        EnsureOrderIsEditable();

        if (quantity <= 0)

```

```

        throw new DomainException("Quantity must be greater than zero.");

        var existingLine = _lines.FirstOrDefault(l => l.ProductId == productId);
        if (existingLine is not null)
        {
            existingLine.IncreaseQuantity(quantity);
        }
        else
        {
            _lines.Add(new OrderLine(productId, productName, unitPrice, quantity));
        }

        RecalculateTotal();
    }

    public void RemoveLine(Guid productId)
    {
        EnsureOrderIsEditable();

        var line = _lines.FirstOrDefault(l => l.ProductId == productId)
            ?? throw new DomainException("Order line not found.");

        _lines.Remove(line);
        RecalculateTotal();
    }

    // Confirming an order is a state transition guarded by an invariant:
    // you cannot confirm an order with no lines. This rule can ONLY be
    // enforced here, inside the aggregate root.
    public void Confirm()
    {
        if (Status != OrderStatus.Draft)
            throw new DomainException($"Cannot confirm an order in status '{Status}'.");

        if (!_lines.Any())
            throw new DomainException("Cannot confirm an order with no lines.");

        Status = OrderStatus.Confirmed;
        _domainEvents.Add(new OrderConfirmedEvent(Id, CustomerId, Total));
    }

    public void ClearDomainEvents() => _domainEvents.Clear();

    private void RecalculateTotal()
    {
        Total = _lines.Aggregate(
            Money.Zero(Total.Currency),
            (sum, line) => sum.Add(line.LineTotal));
    }

    private void EnsureOrderIsEditable()
    {
        if (Status != OrderStatus.Draft)
            throw new DomainException("Only draft orders can be modified.");
    }
}

```

```
// =====
// INTERNAL ENTITY: OrderLine
// Has its own identity, but is only ever accessed THROUGH the
// Order aggregate root -- never fetched or saved on its own.
// =====
public sealed class OrderLine : Entity
{
    public Guid ProductId { get; private set; }
    public string ProductName { get; private set; }
    public Money UnitPrice { get; private set; }
    public int Quantity { get; private set; }
    public Money LineTotal => UnitPrice.Multiply(Quantity);

    // 'internal' constructor: only code inside this assembly (i.e. the
    // Order aggregate itself) can create an OrderLine directly.
    internal OrderLine(Guid productId, string productName, Money unitPrice, int quantity)
    {
        ProductId = productId;
        ProductName = productName;
        UnitPrice = unitPrice;
        Quantity = quantity;
    }

    internal void IncreaseQuantity(int amount) => Quantity += amount;
}
```

3.4 Domain Events

A **Domain Event** represents something meaningful that happened in the domain, expressed in the past tense (`OrderConfirmedEvent`, `CustomerCreditLimitExceededEvent`). They let an aggregate announce a fact without knowing — or caring — who else might need to react to it. This is the **Observer pattern** (Chapter 6.8) applied at the domain level, and it is one of the most powerful tools for keeping aggregates decoupled from each other.

Why not just call the other logic directly?

If `Order.Confirm()` directly called `_inventoryService.ReserveStock()` and `_emailService.SendConfirmation()`, the Order aggregate would depend on infrastructure concerns that have nothing to do with "being an order." Domain Events invert this: the aggregate just says "I was confirmed," and separate, independent handlers (in the Application layer) decide what should happen next. New behavior can be added later by adding a new handler — with **zero changes** to the Order aggregate itself (the Open/Closed Principle in action).

```

// The base contract every Domain Event implements.
public interface IDomainEvent
{
    DateTime OccurredOn { get; }
}

// A concrete event: "an order was created." Immutable record --
// events describe something that already happened and can never change.
public sealed record OrderCreatedEvent(Guid OrderId, Guid CustomerId) : IDomainEvent
{
    public DateTime OccurredOn { get; } = DateTime.UtcNow;
}

public sealed record OrderConfirmedEvent(Guid OrderId, Guid CustomerId, Money Total) : IDomainEvent
{
    public DateTime OccurredOn { get; } = DateTime.UtcNow;
}

// -----
// A handler that reacts to OrderConfirmedEvent. This lives in the
// APPLICATION layer -- never inside the Domain layer -- because
// sending an email is an infrastructure concern, not a domain rule.
// -----
public sealed class SendOrderConfirmationEmailHandler
    : INotificationHandler<DomainEventNotification<OrderConfirmedEvent>>
{
    private readonly IEmailSender _emailSender;

    public SendOrderConfirmationEmailHandler(IEmailSender emailSender)
    {
        _emailSender = emailSender;
    }

    public async Task Handle(
        DomainEventNotification<OrderConfirmedEvent> notification,
        CancellationToken cancellation_token)
    {
        var domainEvent = notification.DomainEvent;
        await _emailSender.SendOrderConfirmationAsync(
            domainEvent.CustomerId, domainEvent.OrderId, domainEvent.Total);
    }
}

```

3.5 Domain Services

Most behavior belongs inside an Entity or Value Object. But sometimes a piece of domain logic naturally involves **multiple aggregates** and does not conceptually belong to any single one of them. That logic goes into a **Domain Service** — a stateless class, still part of the Domain layer, that expresses a domain concept as an operation rather than as an object.

Rule of thumb

Try to put behavior on an Entity or Value Object first. Only reach for a Domain Service when the operation genuinely does not belong to one object — for example, "transfer money between two `Account` aggregates" or "check whether a proposed shipment date conflicts with another order for the same customer."

```
// A Domain Service: the operation involves TWO aggregates (both Accounts),
// so it doesn't naturally belong to either one individually.
public interface IMoneyTransferService
{
    void Transfer(Account source, Account destination, Money amount);
}

public sealed class MoneyTransferService : IMoneyTransferService
{
    // Notice: no repositories, no database calls, no HTTP calls here.
    // A Domain Service only orchestrates pure domain logic and objects
    // that are already loaded in memory -- it stays framework-free.
    public void Transfer(Account source, Account destination, Money amount)
    {
        if (source.Currency != destination.Currency)
            throw new DomainException("Cannot transfer between different currencies.");

        source.Withdraw(amount); // enforces source's own invariants (e.g. sufficient
                                // balance)
        destination.Deposit(amount); // enforces destination's own invariants (e.g. max
                                    // balance)
    }
}
```

3.6 Repositories

A **Repository** gives the illusion of an in-memory collection of Aggregate Roots, hiding all persistence details (SQL, EF Core, MongoDB drivers, HTTP calls, whatever it is) behind a simple interface. The Domain (and Application) layers depend only on the **interface**; the actual implementation lives in the Infrastructure layer. This is the **Dependency Inversion Principle** at work, and it is central to Clean Architecture (Chapter 5).

- One repository per Aggregate Root — never per table, and never per internal entity.
- Repository methods are named using domain language (`GetActiveOrdersForCustomer`), not generic CRUD jargon.
- The interface lives in the **Domain** (or Application) layer; the implementation lives in **Infrastructure**.


```
// Lives in the DOMAIN layer. Notice it has NO EF Core, NO SQL --
// nothing but plain C# and domain types. This is what allows the
// Domain project to have zero dependency on any database technology.
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(Guid id, CancellationToken cancellationToken = default);
    Task<IReadOnlyList<Order>> GetActiveOrdersForCustomerAsync(
        Guid customerId, CancellationToken cancellationToken = default);
    void Add(Order order);
    void Update(Order order);
}
```

3.7 Factories

When creating an object is non-trivial (it requires validation, choosing between subtypes, or assembling several parts), that responsibility is pulled out into a **Factory** — either a static factory method on the aggregate itself (as seen with `Order.Create()` in section 3.3) or a dedicated Factory class when construction is complex enough to deserve its own type. This guarantees an object can never exist in an invalid state, even for a single instant. See Chapter 6.3 for the full Factory Method pattern discussion.

3.8 Specifications

The **Specification pattern** encapsulates a business rule or a query predicate as an object, so it can be named, reused, combined, and tested independently — instead of being written as an inline LINQ expression scattered across the codebase. It answers questions like "does this order qualify as overdue?" or "which repository query represents active premium customers?"

```

// A generic base for building composable Specifications.
public abstract class Specification<T>
{
    public abstract Expression<Func<T, bool>> ToExpression();

    public bool IsSatisfiedBy(T entity) => ToExpression().Compile()(entity);

    // Combine two specifications with AND -- specifications can be
    // composed like building blocks, which is the whole point.
    public Specification<T> And(Specification<T> other) =>
        new AndSpecification<T>(this, other);
}

internal sealed class AndSpecification<T> : Specification<T>
{
    private readonly Specification<T> _left;
    private readonly Specification<T> _right;

    public AndSpecification(Specification<T> left, Specification<T> right)
    {
        _left = left;
        _right = right;
    }

    public override Expression<Func<T, bool>> ToExpression()
    {
        var leftExpr = _left.ToExpression();
        var rightExpr = _right.ToExpression();
        var param = Expression.Parameter(typeof(T));

        var body = Expression.AndAlso(
            Expression.Invoke(leftExpr, param),
            Expression.Invoke(rightExpr, param));

        return Expression.Lambda<Func<T, bool>>(body, param);
    }
}

// A concrete, named business rule: "this order is overdue for shipment."
// Notice how expressive the calling code becomes:
// var overdue = orders.Where(new OverdueOrderSpecification().ToExpression().Compile());
public sealed class OverdueOrderSpecification : Specification<Order>
{
    public override Expression<Func<Order, bool>> ToExpression()
    {
        var cutoff = DateTime.UtcNow.AddDays(-3);
        return order => order.Status == OrderStatus.Confirmed
            && order.ConfirmedOn < cutoff;
    }
}

```

3.9 Tactical Building Blocks — Summary

Block	Has Identity?	Mutable?	Typical Location
Entity	Yes	Yes	Domain layer
Value Object	No	No (immutable)	Domain layer
Aggregate Root	Yes	Yes (via its own methods only)	Domain layer
Domain Event	No	No (immutable fact)	Domain layer (interface) / Application (handlers)
Domain Service	N/A (stateless)	N/A	Domain layer
Repository	N/A	N/A	Interface in Domain/Application, implementation in Infrastructure
Factory	N/A	N/A	Domain layer (often a static method on the aggregate)
Specification	N/A	No (immutable rule)	Domain layer

Chapter 4 — OOP Principles Inside DDD

DDD is not a replacement for Object-Oriented Programming — it is OOP applied with discipline and a clear purpose. Every one of the four pillars of OOP maps directly onto a DDD concept you already saw in Chapter 3.

4.1 Encapsulation

Encapsulation means hiding internal state and only allowing it to change through controlled, validated behavior. In DDD this is enforced aggressively:

- Setters are `private` or `protected` — never public auto-properties like `public string Status { get; set; }`.
- Collections are exposed as `ICollection<T>`, never as a mutable `List<T>` that outside code could clear or manipulate directly.
- State only changes through named methods (`Confirm()`, `AddLine()`) that validate every invariant before applying the change.

Compare the two designs below — both represent the same data, but only one actually protects the business rule.

```
// ANEMIC MODEL -- AVOID THIS.
// Nothing stops external code from setting an impossible state, e.g.
// order.Status = OrderStatus.Shipped; order.Lines.Clear();
public class Order
{
    public Guid Id { get; set; }
    public OrderStatus Status { get; set; }
    public List<OrderLine> Lines { get; set; } = new();
    public decimal Total { get; set; }
}

// All logic ends up dumped into a "manager" class, far away
// from the data it operates on -- classic anemic model symptom.
public class OrderManager
{
    public void ConfirmOrder(Order order)
    {
        // Nothing enforces that THIS is the only path to confirming an order.
        // Any other class anywhere in the codebase could just do
        // order.Status = OrderStatus.Confirmed; and skip validation entirely.
        if (order.Lines.Count == 0)
            throw new Exception("cannot confirm empty order");
        order.Status = OrderStatus.Confirmed;
    }
}
```

```
// RICH MODEL -- as shown fully in section 3.3.
// Status can ONLY change via Confirm(), and Confirm() ALWAYS
// validates the invariant first. There is no other path in the
// entire codebase that can put this object into an invalid state.
public sealed class Order : Entity, IAggregateRoot
{
    public OrderStatus Status { get; private set; } // no public setter

    public void Confirm()
    {
        if (!_lines.Any())
            throw new DomainException("Cannot confirm an order with no lines.");
        Status = OrderStatus.Confirmed;
    }
}
```

4.2 Abstraction

Abstraction means exposing only what matters and hiding implementation detail behind an interface. In DDD this is how the Domain layer stays completely ignorant of infrastructure: it talks to `IOrderRepository`, `IEmailSender`, `IPaymentGateway` — interfaces defined in terms of the business — and never to `SqlConnection`, `SmtpClient`, or a specific payment provider's SDK. Chapter 3.6 (Repositories) is the clearest example of this principle.

4.3 Inheritance

Used sparingly and deliberately in DDD — typically for genuine "is-a" relationships in the domain, such as different types of a concept sharing common behavior. The `Entity` base class from section 3.1 is a good example: every Entity in the system *is* a thing with identity, so sharing that behavior through inheritance is appropriate. DDD favors **composition over inheritance** for everything else — e.g. an `Order` is composed of `OrderLine` and `Money`, it does not inherit from them.

4.4 Polymorphism

Polymorphism lets different concrete types be treated uniformly through a shared interface or base type, with each type providing its own specific behavior. This underlies the Strategy pattern (Chapter 6.5), Repository implementations swapped at runtime via Dependency Injection, and modeling different variations of a domain concept.

```

// Different discount strategies -- each a polymorphic implementation
// of the same domain-defined contract. The Order aggregate (or an
// Application handler) can use ANY of them without knowing which
// concrete type it received -- that decision is made by the
// Dependency Injection container at startup (see Chapter 8.4/8.5).
public interface IDiscountStrategy
{
    Money Apply(Money originalPrice);
}

public sealed class NoDiscount : IDiscountStrategy
{
    public Money Apply(Money originalPrice) => originalPrice;
}

public sealed class PercentageDiscount : IDiscountStrategy
{
    private readonly decimal _percentage;
    public PercentageDiscount(decimal percentage) => _percentage = percentage;

    public Money Apply(Money originalPrice) =>
        new(originalPrice.Amount * (1 - _percentage / 100m), originalPrice.Currency);
}

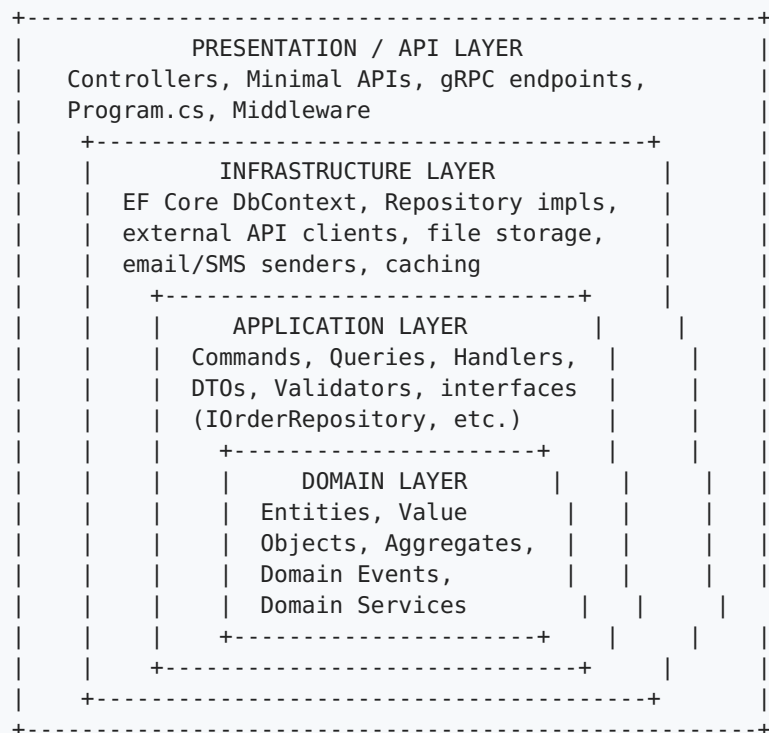
public sealed class LoyaltyCustomerDiscount : IDiscountStrategy
{
    public Money Apply(Money originalPrice) =>
        new(originalPrice.Amount * 0.90m, originalPrice.Currency); // flat 10%
}

```

Chapter 5 — Clean Architecture

5.1 The Dependency Rule

Clean Architecture organizes a solution into concentric layers. The single rule that makes it "clean" is: **source code dependencies can only point inward**. Inner layers know nothing about outer layers — not even their existence.



DEPENDENCY DIRECTION: Presentation -> Infrastructure -> Application -> Domain
The Domain layer has ZERO outgoing dependencies on any other layer.

5.2 The Four Layers, One by One

Domain Layer (the core)

- Contains: Entities, Value Objects, Aggregates, Domain Events, Domain Services, Repository *interfaces*, domain exceptions, Specifications.
- References: **nothing**. No NuGet packages beyond the .NET base class library. No EF Core, no ASP.NET Core, no MediatR.
- This is where the Ubiquitous Language and every tactical pattern from Chapter 3 lives.

Application Layer

- Contains: Use cases, expressed as Commands and Queries (CQRS, Chapter 7), their Handlers, DTOs, validators (FluentValidation), and interfaces for anything infrastructure needs to provide (`IUnitOfWork` , `ISender`).
- References: the Domain layer only. It orchestrates domain objects to fulfill a use case, but contains no business rules itself — those stay inside the Domain layer's entities.
- This layer answers "what can a user/system do?" (place an order, cancel a booking, register a customer) as opposed to "what are the rules?" (the Domain layer's job).

Infrastructure Layer

- Contains: EF Core `DbContext` and entity configurations, concrete Repository implementations, external API clients, file storage, email/SMS senders, caching, and the Unit of Work implementation.
- References: Application and Domain (to implement their interfaces). This is where every "detail" lives — and details are, by design, the easiest part of the system to change or replace.

Presentation Layer (API / Web / Console)

- Contains: ASP.NET Core Controllers or Minimal API endpoints, request/response models, middleware, authentication/authorization wiring, and `Program.cs` (Dependency Injection composition root).
- References: Application (to send Commands/Queries) and Infrastructure (only in `Program.cs` , to register services) — but a controller itself should never call Infrastructure or Domain code directly.

5.3 Why DDD and Clean Architecture Fit So Well Together

DDD Requirement	How Clean Architecture Provides It
The domain model must not depend on databases or frameworks.	The Domain layer is the innermost layer with zero outgoing dependencies — enforced by project references in the .NET solution (the Domain <code>.csproj</code> literally cannot reference EF Core).
Repository interfaces must be defined by the domain, implemented elsewhere.	Dependency Inversion: interfaces live in Domain/Application, concrete classes live in Infrastructure, wired together at startup via Dependency Injection.
Use cases should be explicit and testable independent of UI/DB.	The Application layer's Commands/Queries/Handlers are the use cases — they can be unit tested with in-memory fakes, with no web server or real database involved.
Bounded Contexts should be independently deployable/replaceable.	Each Bounded Context can be its own solution of Domain/Application/Infrastructure/Presentation projects — the architecture scales from a single module to a full microservice.

A simple mental test

"If I deleted my Infrastructure and Presentation projects entirely and started them from scratch with a totally different database and a totally different UI framework, would a single line inside my Domain project need to change?" In a correctly-built Clean Architecture + DDD solution, the answer is always **no**.

Chapter 6 — Design Patterns Used Across the Architecture

DDD does not exist in isolation — it is normally implemented using a set of well-known Gang-of-Four and enterprise patterns. This chapter catalogs the ones you will meet constantly in a .NET DDD codebase, several of which you already saw applied in Chapters 3–4.

6.1 Repository Pattern

Already introduced in section 3.6. Provides a collection-like interface over persistence, hiding EF Core or any other storage mechanism from the Domain and Application layers.

6.2 Unit of Work Pattern

Tracks all changes made across one or more repositories during a single business transaction and commits them together, atomically. In EF Core, `DbContext` already behaves like a Unit of Work internally, so the pattern is usually implemented as a thin wrapper around `SaveChangesAsync()`.

```
// Interface -- lives in the Application layer.
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(CancellationTokentoken cancellationTokentoken = default);
}

// Implementation -- lives in the Infrastructure layer, wraps EF Core's
// own change-tracking + transaction behavior.
public sealed class EfUnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;
    public EfUnitOfWork(AppDbContext context) => _context = context;

    public Task<int> SaveChangesAsync(CancellationTokentoken cancellationTokentoken = default) =>
        _context.SaveChanges(cancellationTokentoken);
}
```

6.3 Factory Method Pattern

Already applied in section 3.3 (`Order.Create()`). Centralizes and hides the logic of constructing a valid object, especially when construction requires validation or decisions about which concrete type to build.

6.4 Specification Pattern

Already covered fully in section 3.8. Encapsulates a reusable business rule or query predicate as a named, composable object.

6.5 Strategy Pattern

Already applied in section 4.4 (`IDiscountStrategy`). Lets an algorithm vary independently from the code that uses it, by extracting each variant behind a common interface and selecting the concrete implementation at runtime (often via Dependency Injection).

6.6 Mediator Pattern (and CQRS)

The Mediator pattern removes direct dependencies between objects by routing all communication through a central mediator object. In .NET, the **MediatR** library is the de-facto standard implementation, and it is the backbone of the CQRS approach detailed fully in Chapter 7: instead of a controller directly calling five different services, it sends one message (`mediator.Send(command)`) and MediatR routes it to exactly one handler.

6.7 Decorator Pattern

Wraps an object with another object implementing the same interface, adding behavior transparently. In DDD-based .NET solutions this is extremely common for cross-cutting concerns around Application layer handlers — logging, validation, caching, and transaction management are frequently implemented as MediatR *Pipeline Behaviors*, which are essentially decorators around every handler.

```
// A MediatR Pipeline Behavior -- a Decorator that wraps EVERY command/query
// handler in the application to add logging, without any handler needing
// to know logging exists. This keeps cross-cutting concerns out of
// business logic entirely.
public sealed class LoggingBehavior<TRequest, TResponse>
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull
{
    private readonly ILogger<LoggingBehavior<TRequest, TResponse>> _logger;

    public LoggingBehavior(ILogger<LoggingBehavior<TRequest, TResponse>> logger)
        => _logger = logger;

    public async Task<TResponse> Handle(
        TRequest request,
        RequestHandlerDelegate<TResponse> next,
        CancellationToken cancellationToken)
    {
        _logger.LogInformation("Handling {RequestName}", typeof(TRequest).Name);
        var response = await next(); // calls the actual handler (or the next decorator)
        _logger.LogInformation("Handled {RequestName}", typeof(TRequest).Name);
        return response;
    }
}
```

6.8 Observer Pattern

Already applied in section 3.4 (Domain Events). One object (the aggregate) publishes a notification; any number of independent observers (event handlers) subscribe and react, without the publisher knowing who is listening or how many observers exist.

6.9 Builder Pattern

Used when constructing a complex object (or a complex test fixture) step by step. Common in unit tests for building aggregates in a specific state without duplicating setup code across many test methods (see Chapter 9).

6.10 Adapter Pattern (Anti-Corruption Layer)

Converts one interface into another that the client expects — this is exactly the mechanism behind the **Anti-Corruption Layer** from section 2.4. When integrating a third-party payment provider's SDK, you write an adapter implementing your own domain-defined `IPaymentGateway` interface, translating your domain concepts into whatever shape the external SDK expects (and vice-versa), so the external vendor's model never leaks into your domain.

6.11 Pattern Cheat Sheet

Pattern	Solves	Where It Lives
Repository	Hides persistence behind a collection-like interface	Domain (interface) / Infrastructure (impl.)
Unit of Work	Commits multiple changes atomically	Application (interface) / Infrastructure (impl.)
Factory Method	Guarantees valid object construction	Domain
Specification	Reusable, composable business rules/queries	Domain
Strategy	Interchangeable algorithms/behaviors	Domain or Application
Mediator (CQRS)	Decouples callers from handlers	Application
Decorator (Pipeline Behavior)	Cross-cutting concerns (logging, validation)	Application
Observer (Domain Events)	Decoupled reactions to domain facts	Domain (event) / Application (handler)
Builder	Step-by-step complex object construction	Tests, sometimes Domain
Adapter (ACL)	Isolates external/legacy models from the domain	Infrastructure

Chapter 7 — CQRS and MediatR in Practice

7.1 What CQRS Actually Means

Command Query Responsibility Segregation (CQRS) is the principle that operations which **change state** (Commands) should be modeled and handled separately from operations that **read state** (Queries). This does not require two databases or event sourcing — that is an optional, advanced extreme. In most .NET DDD solutions, CQRS simply means:

- A `CreateOrderCommand` is a different class from a `GetOrderByIdQuery`, each with its own dedicated handler.
- Commands go through the rich domain model (Aggregates) to guarantee invariants are enforced.
- Queries can bypass the domain model entirely and read directly into lightweight DTOs for performance — there is no need to load a full `Order` aggregate with all its behavior just to display an order summary on a screen.

7.2 Why MediatR

MediatR implements the Mediator pattern (section 6.6): controllers and other callers do not depend on concrete handler classes at all — they just send a message and MediatR finds the one handler registered for it via Dependency Injection. Benefits:

- **Thin controllers:** a controller action becomes 2–3 lines — build the command/query, send it, return the result.
- **Single Responsibility:** each handler does exactly one use case, making the codebase easy to navigate (one file per use case, easy to find and reason about).
- **Pluggable cross-cutting concerns:** via Pipeline Behaviors (section 6.7) for logging, validation, transactions, caching — applied uniformly with no repeated boilerplate per handler.



The full working example — Commands, Queries, Handlers, and Validators — is built end-to-end in Chapter 8.3 as part of the complete Order Management project.

Chapter 8 — Full Practical Project: Online Store Order Management

This chapter ties every concept from Chapters 2–7 together into one coherent, working slice of a real system: placing and confirming orders in an online store. Every layer is shown with full code and inline comments. The domain code reuses and extends the `Order` aggregate built throughout Chapter 3.

8.1 Solution & Folder Structure

A typical .NET solution implementing DDD + Clean Architecture is split into one project per layer, with project references only allowed to point inward, exactly matching the Dependency Rule from Chapter 5.

```

OnlineStore.sln
|
+-- src/
|   +-- OnlineStore.Domain/                (no project references -- innermost layer)
|   |   +-- Common/
|   |   |   +-- Entity.cs
|   |   |   +-- IAggregateRoot.cs
|   |   |   +-- IDomainEvent.cs
|   |   |   +-- DomainException.cs
|   |   +-- Orders/
|   |   |   +-- Order.cs
|   |   |   +-- OrderLine.cs
|   |   |   +-- OrderStatus.cs
|   |   |   +-- Events/
|   |   |   |   +-- OrderCreatedEvent.cs
|   |   |   |   +-- OrderConfirmedEvent.cs
|   |   |   +-- IOrderRepository.cs
|   |   +-- ValueObjects/
|   |   |   +-- Money.cs
|   |
|   +-- OnlineStore.Application/            (references: Domain)
|   |   +-- Common/
|   |   |   +-- IUnitOfWork.cs
|   |   |   +-- Behaviors/
|   |   |   |   +-- ValidationBehavior.cs
|   |   |   |   +-- LoggingBehavior.cs
|   |   +-- Orders/
|   |   |   +-- Commands/
|   |   |   |   +-- CreateOrder/
|   |   |   |   |   +-- CreateOrderCommand.cs
|   |   |   |   |   +-- CreateOrderCommandHandler.cs
|   |   |   |   |   +-- CreateOrderCommandValidator.cs
|   |   |   |   +-- ConfirmOrder/
|   |   |   |   |   +-- ConfirmOrderCommand.cs
|   |   |   |   |   +-- ConfirmOrderCommandHandler.cs
|   |   |   +-- Queries/
|   |   |   |   +-- GetOrderById/
|   |   |   |   |   +-- GetOrderByIdQuery.cs
|   |   |   |   |   +-- GetOrderByIdQueryHandler.cs
|   |   |   |   +-- OrderDto.cs
|   |
|   +-- OnlineStore.Infrastructure/         (references: Application, Domain)
|   |   +-- Persistence/
|   |   |   +-- AppDbContext.cs
|   |   |   +-- Configurations/
|   |   |   |   +-- OrderConfiguration.cs
|   |   |   +-- Repositories/
|   |   |   |   +-- OrderRepository.cs
|   |   +-- EfUnitOfWork.cs
|   |   +-- DependencyInjection.cs
|   |
|   +-- OnlineStore.Api/                   (references: Application, Infrastructure)
|   |   +-- Controllers/
|   |   |   +-- OrdersController.cs
|   |   +-- Middleware/

```



```
|
|      +-- ExceptionHandlingMiddleware.cs
|      +-- Program.cs
|
+-- tests/
|     +-- OnlineStore.Domain.Tests/
|         +-- OrderTests.cs
```

8.2 Domain Layer

The `Entity`, `Money`, `Order`, `OrderLine` and Domain Event classes were already built in full in Chapter 3 (sections 3.1–3.4) — they belong to `OnlineStore.Domain` unchanged. Two small supporting pieces complete the layer:

```
public enum OrderStatus
{
    Draft = 0,
    Confirmed = 1,
    Shipped = 2,
    Cancelled = 3
}

// A dedicated exception type for business rule violations.
// Distinguishing DomainException from generic exceptions lets the
// Presentation layer map broken business rules to HTTP 400 (Bad Request),
// while unexpected bugs still surface as HTTP 500 -- see section 8.5.
public sealed class DomainException : Exception
{
    public DomainException(string message) : base(message) { }
}
```

8.3 Application Layer — CQRS with MediatR

This is where use cases live. Each use case is a small, self-contained folder containing a Command (or Query), its Handler, and its Validator — a structure often called **Vertical Slice** organization, which pairs naturally with CQRS.

8.3.1 Command: Create Order

```
// The Command -- an immutable request object describing intent.
// MediatR's IRequest<TResponse> marks it as something MediatR can route.
public sealed record CreateOrderCommand(
    Guid CustomerId,
    string Currency,
    List<CreateOrderLineDto> Lines) : IRequest<Guid>;

public sealed record CreateOrderLineDto(
    Guid ProductId, string ProductName, decimal UnitPrice, int Quantity);
```

```

// FluentValidation validator. Runs automatically inside a MediatR
// Pipeline Behavior (a Decorator, section 6.7) BEFORE the handler
// executes -- the handler itself never has to check "is this input
// well-formed", only "is this a valid business operation".
public sealed class CreateOrderCommandValidator : AbstractValidator<CreateOrderCommand>
{
    public CreateOrderCommandValidator()
    {
        RuleFor(x => x.CustomerId).NotEmpty();
        RuleFor(x => x.Currency).Length(3);
        RuleFor(x => x.Lines).NotEmpty()
            .WithMessage("An order must contain at least one line.");

        RuleForEach(x => x.Lines).ChildRules(line =>
        {
            line.RuleFor(l => l.ProductId).NotEmpty();
            line.RuleFor(l => l.ProductName).NotEmpty();
            line.RuleFor(l => l.UnitPrice).GreaterThan(0);
            line.RuleFor(l => l.Quantity).GreaterThan(0);
        });
    }
}

```

```

// The Handler -- orchestrates the use case. It does NOT contain
// business rules itself (those live in the Order aggregate); it only
// coordinates: build the aggregate, call its behavior, persist it.
public sealed class CreateOrderCommandHandler : IRequestHandler<CreateOrderCommand, Guid>
{
    private readonly IOrderRepository _orderRepository;
    private readonly IUnitOfWork _unitOfWork;

    public CreateOrderCommandHandler(IOrderRepository orderRepository, IUnitOfWork unitOfWork)
    {
        _orderRepository = orderRepository;
        _unitOfWork = unitOfWork;
    }

    public async Task<Guid> Handle(CreateOrderCommand request, CancellationToken cancellationToken)
    {
        // Delegates creation to the aggregate's own Factory Method (section 3.3/6.3) --
        // the handler never constructs an Order "by hand" with 'new Order(...)'.
        var order = Order.Create(request.CustomerId, request.Currency);

        foreach (var line in request.Lines)
        {
            var unitPrice = new Money(line.UnitPrice, request.Currency);
            order.AddLine(line.ProductId, line.ProductName, unitPrice, line.Quantity);
        }

        _orderRepository.Add(order);

        // SaveChangesAsync persists the aggregate AND (via an interceptor,
        // see the note below) dispatches any Domain Events it raised.
        await _unitOfWork.SaveChangesAsync(cancellationToken);

        return order.Id;
    }
}

```

8.3.2 Command: Confirm Order

```
public sealed record ConfirmOrderCommand(Guid OrderId) : IRequest;

public sealed class ConfirmOrderCommandHandler : IRequestHandler<ConfirmOrderCommand>
{
    private readonly IOrderRepository _orderRepository;
    private readonly IUnitOfWork _unitOfWork;

    public ConfirmOrderCommandHandler(IOrderRepository orderRepository, IUnitOfWork unitOfWork)
    {
        _orderRepository = orderRepository;
        _unitOfWork = unitOfWork;
    }

    public async Task Handle(ConfirmOrderCommand request, CancellationToken cancellationToken)
    {
        var order = await _orderRepository.GetByIdAsync(request.OrderId, cancellationToken)
            ?? throw new NotFoundException($"Order {request.OrderId} was not found.");

        // The ENTIRE business rule ("cannot confirm an empty order", etc.)
        // is enforced inside this single method call -- the handler trusts
        // the aggregate completely and adds no business logic of its own.
        order.Confirm();

        await _unitOfWork.SaveChangesAsync(cancellationToken);
    }
}
```

8.3.3 Query: Get Order By Id

Queries are intentionally simpler than Commands. They do not need to load a full `Order` aggregate with all its private state and behavior — that would be wasteful. Instead, they can query the database directly and project straight into a DTO, which is one of the biggest performance benefits of CQRS.

```

public sealed record GetOrderByIdQuery(Guid OrderId) : IRequest<OrderDto?>;

// A flat, read-only shape designed purely for the client -- no domain
// behavior, no invariants, just data. This is deliberately DIFFERENT
// from the Order aggregate: queries and commands can each have the
// shape that best suits their purpose (the essence of CQRS).
public sealed record OrderDto(
    Guid Id,
    Guid CustomerId,
    string Status,
    decimal Total,
    string Currency,
    List<OrderLineDto> Lines);

public sealed record OrderLineDto(
    Guid ProductId, string ProductName, decimal UnitPrice, int Quantity);

public sealed class GetOrderByIdQueryHandler : IRequestHandler<GetOrderByIdQuery, OrderDto?>
{
    private readonly AppDbContext _dbContext; // direct read access is fine for queries

    public GetOrderByIdQueryHandler(AppDbContext dbContext) => _dbContext = dbContext;

    public async Task<OrderDto?> Handle(GetOrderByIdQuery request, CancellationToken cancellationToken)
    {
        // AsNoTracking() -- this is a read-only query, so EF Core does not
        // need to track the entities for change detection. Faster and lighter.
        return await _dbContext.Orders
            .AsNoTracking()
            .Where(o => o.Id == request.OrderId)
            .Select(o => new OrderDto(
                o.Id,
                o.CustomerId,
                o.Status.ToString(),
                o.Total.Amount,
                o.Total.Currency,
                o.Lines.Select(l => new OrderLineDto(
                    l.ProductId, l.ProductName, l.UnitPrice.Amount, l.Quantity)).ToList()))
            .FirstOrDefaultAsync(cancellationToken);
    }
}

```

8.4 Infrastructure Layer

This layer implements every interface the Domain and Application layers defined, and contains all framework-specific code: EF Core mapping, the concrete repository, and Dependency Injection wiring.

8.4.1 EF Core DbContext & Entity Configuration

A key detail: Value Objects and EF Core

EF Core needs to know how to map a Value Object like `Money` (which has no identity and is not itself a table) into columns on the `Orders` table. This is done with `OwnsOne` in the Fluent API configuration below — it keeps the domain model completely free of any EF Core attributes or annotations.

```
public sealed class AppDbContext : DbContext
{
    public DbSet<Order> Orders => Set<Order>();

    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Applies every IEntityTypeConfiguration<T> found in this assembly --
        // keeps mapping code organized in one file per aggregate instead of
        // one giant OnModelCreating method.
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

```

public sealed class OrderConfiguration : IEntityTypeConfiguration<Order>
{
    public void Configure(EntityTypeBuilder<Order> builder)
    {
        builder.ToTable("Orders");
        builder.HasKey(o => o.Id);

        builder.Property(o => o.Status)
            .HasConversion<string>() // store the enum as text -- readable in the DB
            .HasMaxLength(20);

        // Money is a Value Object with no identity -- OwnsOne tells EF Core
        // to map its properties as columns on the SAME Orders table,
        // instead of treating it as a separate related entity.
        builder.OwnsOne(o => o.Total, money =>
        {
            money.Property(m => m.Amount).HasColumnName("TotalAmount").HasColumnType("decimal(18,2)");
            money.Property(m => m.Currency).HasColumnName("TotalCurrency").HasMaxLength(3);
        });

        // OrderLines are internal entities of the Order aggregate --
        // mapped as an owned collection so they are ALWAYS loaded and
        // saved together with their parent Order, never independently.
        builder.OwnsMany(o => o.Lines, line =>
        {
            line.ToTable("OrderLines");
            line.WithOwner().HasForeignKey("OrderId");
            line.Property<Guid>("Id").ValueGeneratedOnAdd();
            line.HasKey("Id");

            line.OwnsOne(l => l.UnitPrice, money =>
            {
                money.Property(m => m.Amount).HasColumnName("UnitPriceAmount").HasColumnType("decimal(18,2)");
                money.Property(m => m.Currency).HasColumnName("UnitPriceCurrency").HasMaxLength(3);
            });
        });

        // Domain Events must NEVER be persisted -- they are transient
        // messages, not state. Explicitly ignore them in the mapping.
        builder.Ignore(o => o.DomainEvents);
    }
}

```

8.4.2 Repository Implementation

```
// Implements the interface DEFINED in the Domain layer (section 3.6).
// Notice this class lives in Infrastructure, not Domain -- the Domain
// layer never sees EF Core, DbContext, or LINQ-to-Entities specifics.
public sealed class OrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;
    public OrderRepository(AppDbContext context) => _context = context;

    public async Task<Order?> GetByIdAsync(Guid id, CancellationToken cancellationToken = default) =>
        await _context.Orders
            .Include(o => o.Lines)
            .FirstOrDefaultAsync(o => o.Id == id, cancellationToken);

    public async Task<IReadOnlyList<Order>> GetActiveOrdersForCustomerAsync(
        Guid customerId, CancellationToken cancellationToken = default) =>
        await _context.Orders
            .Include(o => o.Lines)
            .Where(o => o.CustomerId == customerId && o.Status != OrderStatus.Cancelled)
            .ToListAsync(cancellationToken);

    public void Add(Order order) => _context.Orders.Add(order);

    // EF Core's change tracker already knows about a loaded, modified
    // aggregate -- Update() is a no-op safeguard for cases where the
    // aggregate might be attached in a detached state.
    public void Update(Order order) => _context.Orders.Update(order);
}
```

8.4.3 Dispatching Domain Events on Save

The final piece connects Chapter 3.4's Domain Events to the real world: right before (or after) `SaveChangesAsync` commits a transaction, every pending Domain Event collected by any tracked aggregate is published through MediatR, triggering the handlers shown in section 3.4.


```

public sealed class EfUnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;
    private readonly IPublisher _publisher; // MediatR's publish-only interface

    public EfUnitOfWork(AppDbContext context, IPublisher publisher)
    {
        _context = context;
        _publisher = publisher;
    }

    public async Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
    {
        // 1. Collect every domain event raised by every tracked aggregate
        // BEFORE saving, since ClearDomainEvents() will wipe them after.
        var aggregatesWithEvents = _context.ChangeTracker
            .Entries<Entity>()
            .Select(e => e.Entity)
            .OfType<IAggregateRoot>()
            .Where(a => ((Entity)a).GetType()
                .GetProperty(nameof(Order.DomainEvents)) is not null)
            .ToList();

        // 2. Persist the actual state changes first.
        var result = await _context.SaveChangesAsync(cancellationToken);

        // 3. THEN publish the events -- guarantees we never notify the
        // rest of the system about something that failed to save.
        foreach (var order in aggregatesWithEvents.OfType<Order>())
        {
            foreach (var domainEvent in order.DomainEvents)
            {
                await _publisher.Publish(
                    new DomainEventNotification<IDomainEvent>(domainEvent), cancellationToken);
            }
            order.ClearDomainEvents();
        }

        return result;
    }
}

// A thin wrapper so any IDomainEvent can be published as a MediatR notification.
public sealed record DomainEventNotification<T>(T DomainEvent) : INotification where T : IDomainEvent;

```

8.4.4 Dependency Injection Registration

This is the composition root for the Infrastructure layer — the one place where interfaces and implementations are finally connected.

```

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services, IConfiguration configuration)
    {
        services.AddDbContext<AppDbContext>(options =>
            options.UseSqlServer(configuration.GetConnectionString("Default")));

        // Every interface defined in Domain/Application gets its concrete
        // Infrastructure implementation registered here -- and ONLY here.
        services.AddScoped<IOrderRepository, OrderRepository>();
        services.AddScoped<IUnitOfWork, EfUnitOfWork>();

        return services;
    }
}

```

8.5 Presentation Layer

8.5.1 API Controller

Notice how thin this controller is: it has no business logic, no database access, and no validation logic of its own. It only translates HTTP into a MediatR message and back.

```

[ApiController]
[Route("api/orders")]
public sealed class OrdersController : ControllerBase
{
    private readonly ISender _mediator; // MediatR's send-only interface

    public OrdersController(ISender mediator) => _mediator = mediator;

    [HttpPost]
    public async Task<ActionResult<Guid>> CreateOrder(
        CreateOrderCommand command, CancellationToken cancellationToken)
    {
        var orderId = await _mediator.Send(command, cancellationToken);
        return CreatedAtAction(nameof(GetOrder), new { id = orderId }, orderId);
    }

    [HttpPost("{id:guid}/confirm")]
    public async Task<IActionResult> ConfirmOrder(Guid id, CancellationToken cancellationToken)
    {
        await _mediator.Send(new ConfirmOrderCommand(id), cancellationToken);
        return NoContent();
    }

    [HttpGet("{id:guid}")]
    public async Task<ActionResult<OrderDto>> GetOrder(Guid id, CancellationToken cancellationToken)
    {
        var order = await _mediator.Send(new GetOrderByIdQuery(id), cancellationToken);
        return order is not null ? Ok(order) : NotFound();
    }
}

```

8.5.2 Global Exception Handling Middleware

Because the Domain layer throws typed exceptions (`DomainException` , `NotFoundException`), the Presentation layer can translate them into correct HTTP status codes in exactly one place, instead of wrapping every controller action in a try/catch block.

```

public sealed class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    public ExceptionHandlingMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (DomainException ex)
        {
            // A broken BUSINESS rule -> 400 Bad Request. The client sent
            // a request that is well-formed but not currently allowed.
            context.Response.StatusCode = StatusCodes.Status400BadRequest;
            await context.Response.WriteAsJsonAsync(new { error = ex.Message });
        }
        catch (NotFoundException ex)
        {
            context.Response.StatusCode = StatusCodes.Status404NotFound;
            await context.Response.WriteAsJsonAsync(new { error = ex.Message });
        }
        catch (ValidationException ex) // FluentValidation
        {
            context.Response.StatusCode = StatusCodes.Status422UnprocessableEntity;
            await context.Response.WriteAsJsonAsync(new { errors = ex.Errors });
        }
        catch (Exception)
        {
            // Anything else is a genuine bug, not a business rule violation.
            context.Response.StatusCode = StatusCodes.Status500InternalServerError;
            await context.Response.WriteAsJsonAsync(new { error = "An unexpected error
occurred." });
        }
    }
}

```

8.5.3 Program.cs — Wiring Everything Together

```
var builder = WebApplication.CreateBuilder(args);

// Presentation-layer services
builder.Services.AddControllers();

// Application-layer services: MediatR scans the assembly and registers
// every IRequestHandler/INotificationHandler it finds automatically.
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(CreateOrderCommand).Assembly));

// Register Pipeline Behaviors (Decorators, section 6.7) -- order matters:
// validation runs before logging wraps the actual handler call.
builder.Services.AddTransient(typeof(IPipelineBehavior<, >), typeof(ValidationBehavior<, >));
builder.Services.AddTransient(typeof(IPipelineBehavior<, >), typeof(LoggingBehavior<, >));

builder.Services.AddValidatorsFromAssembly(typeof(CreateOrderCommandValidator).Assembly);

// Infrastructure-layer services (EF Core, repositories, unit of work)
builder.Services.AddInfrastructure(builder.Configuration);

var app = builder.Build();

app.UseMiddleware<ExceptionHandlerMiddleware>();
app.MapControllers();
app.Run();
```

Chapter 9 — Unit Testing the Domain Layer

One of the biggest practical payoffs of DDD + Clean Architecture is testability. Because the Domain layer has zero dependencies on databases, web frameworks, or external services, its business rules can be unit tested in complete isolation, with no mocking framework needed at all — you are just testing plain C# objects.

```

public class OrderTests
{
    // Naming convention: MethodName_Scenario_ExpectedResult.
    // This makes a failing test's PURPOSE obvious from its name alone.

    [Fact]
    public void Create_WithValidCustomerId_CreatesOrderInDraftStatus()
    {
        // Arrange & Act
        var order = Order.Create(Guid.NewGuid(), "USD");

        // Assert
        Assert.Equal(OrderStatus.Draft, order.Status);
        Assert.Empty(order.Lines);
    }

    [Fact]
    public void Create_WithEmptyCustomerId_ThrowsArgumentException()
    {
        Assert.Throws<ArgumentException>(() => Order.Create(Guid.Empty, "USD"));
    }

    [Fact]
    public void AddLine_WithValidData_IncreasesTotal()
    {
        var order = Order.Create(Guid.NewGuid(), "USD");

        order.AddLine(Guid.NewGuid(), "Keyboard", new Money(50m, "USD"), 2);

        Assert.Equal(100m, order.Total.Amount);
    }

    [Fact]
    public void AddLine_SameProductTwice_MergesIntoOneLineWithCombinedQuantity()
    {
        var order = Order.Create(Guid.NewGuid(), "USD");
        var productId = Guid.NewGuid();

        order.AddLine(productId, "Mouse", new Money(20m, "USD"), 1);
        order.AddLine(productId, "Mouse", new Money(20m, "USD"), 2);

        Assert.Single(order.Lines);
        Assert.Equal(3, order.Lines.First().Quantity);
    }

    [Fact]
    public void Confirm_WithNoLines_ThrowsDomainException()
    {
        var order = Order.Create(Guid.NewGuid(), "USD");

        // This test alone PROVES the invariant from section 3.3 is
        // enforced -- no database, no HTTP server, no framework needed.
        var exception = Assert.Throws<DomainException>(() => order.Confirm());
        Assert.Equal("Cannot confirm an order with no lines.", exception.Message);
    }
}

```

```

[Fact]
public void Confirm_WithAtLeastOneLine_ChangesStatusToConfirmedAndRaisesEvent()
{
    var order = Order.Create(Guid.NewGuid(), "USD");
    order.AddLine(Guid.NewGuid(), "Monitor", new Money(300m, "USD"), 1);

    order.Confirm();

    Assert.Equal(OrderStatus.Confirmed, order.Status);
    Assert.Contains(order.DomainEvents, e => e is OrderConfirmedEvent);
}

[Fact]
public void AddLine_AfterOrderIsConfirmed_ThrowsDomainException()
{
    var order = Order.Create(Guid.NewGuid(), "USD");
    order.AddLine(Guid.NewGuid(), "Cable", new Money(5m, "USD"), 1);
    order.Confirm();

    // Proves the "only draft orders can be modified" invariant --
    // a rule that a purely anemic model (Chapter 4.1) could never guarantee.
    Assert.Throws<DomainException>(() =>
        order.AddLine(Guid.NewGuid(), "Adapter", new Money(10m, "USD"), 1));
}
}

```

Testing pyramid for a DDD solution

- **Domain layer:** pure unit tests, no mocks needed (as above) — the majority of your tests should live here, since this is where the business rules are.
- **Application layer:** unit tests for handlers using in-memory fakes or mocks (e.g. Moq) for `IOrderRepository` / `IUnitOfWork`.
- **Infrastructure/API layer:** a smaller number of integration tests, often using `WebApplicationFactory` and a real (or in-memory/SQLite) database, to prove the wiring is correct end-to-end.

Chapter 10 — When (Not) to Use DDD

DDD is not free. Its patterns add ceremony and abstraction layers that only pay off when the domain complexity justifies them. Applying full DDD to a simple CRUD admin panel is a well-known source of over-engineering.

10.1 Good Fit for DDD

- The business has genuinely complex, evolving rules (insurance underwriting, logistics routing, financial trading, healthcare workflows, complex pricing/promotions engines).
- The system is expected to live and grow for years, with a team that needs a shared, precise language with domain experts.
- Multiple distinct sub-businesses exist inside one system that would benefit from being modeled separately (justifying Bounded Contexts).
- Correctness of business rules matters more than raw development speed on day one.

10.2 Poor Fit for DDD

- Simple CRUD applications with little to no business logic beyond validation (e.g. a basic internal admin tool that just lists/edits records).
- Short-lived prototypes, MVPs, or throwaway scripts where the cost of ceremony outweighs any long-term benefit.
- Systems whose entire complexity is technical (e.g. high-throughput data pipelines) rather than about business rules.

A pragmatic middle ground

It is entirely valid — and common in real .NET codebases — to apply full tactical DDD (rich aggregates, value objects, domain events) only to the Core Domain (section 2.3), while using simple, mostly anemic CRUD services with EF Core directly for Generic and Supporting subdomains such as user profile settings or static content pages. DDD is a tool to apply *where it earns its cost*, not a rule to apply uniformly to every class in a solution.

Chapter 11 — Common Pitfalls & Anti-Patterns

11.1 Anemic Domain Model

Covered in section 4.1. Classes are just data bags; all logic lives in external "Service" or "Manager" classes. This is the single most common way teams claim to "do DDD" while getting none of its benefits.

11.2 God Aggregates

Aggregates that try to model too much (e.g. a `Customer` aggregate containing every order, invoice, support ticket, and login session ever recorded). Symptoms: slow loads, frequent concurrency conflicts (optimistic concurrency exceptions), and huge, hard-to-review pull requests. Fix: keep aggregates small, referencing other aggregates only by Id.

11.3 Leaking Infrastructure into the Domain

Adding EF Core attributes (`[Column]` , `[Table]`), `JsonPropertyName` attributes, or a reference to `Microsoft.EntityFrameworkCore` inside the Domain project. This silently breaks the Dependency Rule from Chapter 5 — always map persistence details through Fluent API configuration classes in Infrastructure instead (as shown in section 8.4.1).

11.4 Repository Per Table Instead of Per Aggregate

Creating `IOrderLineRepository` alongside `IOrderRepository` violates the Aggregate rule from section 3.3 — internal entities should never be fetched or saved independently of their Aggregate Root.

11.5 Treating Bounded Contexts as Just "Microservices"

A Bounded Context is a *modeling* boundary, not necessarily a network/deployment boundary. Many teams split into microservices prematurely, based on team structure rather than actual model boundaries, and pay a heavy distributed-systems tax for no real domain benefit. Start with well-separated modules inside one deployable; split into services later if and when it is actually justified.

11.6 Overusing Domain Events for Everything

Domain Events are excellent for decoupling reactions to a fact, but chaining many events triggering more events can make a codebase very difficult to trace and debug. Use them for genuine "this happened, and other unrelated things should now happen" scenarios — not as a general replacement for simple, direct method calls within the same use case.

11.7 Validation Logic Duplicated Across Layers

Format validation (e.g. "email must contain @") belongs in Value Objects/FluentValidation, checked once. Business rule validation (e.g. "cannot confirm an order with no lines") belongs exclusively inside the

Aggregate. A common mistake is re-checking business rules again in the Application handler or even in the Controller "just to be safe" — this duplicates logic and inevitably drifts out of sync over time.

Chapter 12 — Summary & Further Reading

12.1 The Big Picture, One Page

Concept	One-Line Summary
Ubiquitous Language	One precise vocabulary shared by code and business.
Bounded Context	A boundary within which one model and language apply consistently.
Entity	Identity-based object; mutable; equal by Id.
Value Object	Immutable object defined entirely by its values.
Aggregate Root	The single gatekeeper for all changes to a cluster of objects.
Domain Event	An immutable fact the aggregate announces, decoupling reactions.
Repository	Collection-like abstraction hiding persistence details.
Clean Architecture	Layered structure enforcing that dependencies always point inward, toward the Domain.
CQRS + MediatR	Separate, explicit use cases for reads and writes, routed through a mediator.

12.2 A Practical Path to Learning DDD Deeply

1. Re-read Chapter 8 and re-type the code by hand in a real .NET solution — typing builds muscle memory that reading alone does not.
2. Extend the Order Management example: add a `Product` aggregate with stock levels, and enforce "cannot confirm an order if any line exceeds available stock" as a new invariant.
3. Introduce a second Bounded Context (e.g. **Shipping**) that reacts to `OrderConfirmedEvent` via an integration event, to practice Context Mapping (section 2.4) for real.
4. Write the Domain layer unit tests (Chapter 9) *before* writing the Infrastructure and API layers, to feel how a rich domain model can be fully specified and verified in isolation.

12.3 Recommended Further Reading

- *Domain-Driven Design: Tackling Complexity in the Heart of Software* — Eric Evans (the original, foundational text).
- *Implementing Domain-Driven Design* — Vaughn Vernon (the practical, code-heavy follow-up; excellent for translating theory into working systems).
- *Clean Architecture: A Craftsman's Guide to Software Structure and Design* — Robert C. Martin.
- Microsoft's official ".NET Microservices: Architecture for Containerized .NET Applications" e-book (free), which documents a full open-source DDD + Clean Architecture reference implementation ("eShopOnContainers"/"eShop") in ASP.NET Core.

Final note

DDD is a way of thinking before it is a way of coding. The patterns in this guide are tools for expressing a deep understanding of the business in code — but that deep understanding itself, built through constant conversation with domain experts, is the actual goal. Code and diagrams are just the byproduct.